



cs695-assignment2

Updated automatically every 5 minutes

Assignment #2

CS695 Spring 2024-25

virtual is real

Statutory note:

- This course is on a ***no-plagiarism*** diet.
- All parties involved in plagiarism harakiri will be penalized to the maximum extent.
- The [Moss](#), [JPlag](#), and [Dolos](#) Detective Agencies have agreed to conduct all investigations.
- Byomkesh, Hercule, Phryne, Marple, Enola, and Sherlock are on standby.
- Hardcoding the runtime output in the code will be heavily penalized.
- Generative AI (DeepSeek R1, ChatGPT, Gemini, etc.) is your friend, but it cannot generate outputs for you to submit. If you want to submit such generated outputs, mark them explicitly in your submissions.
- **Submission guidelines to be strictly followed; else will not be graded.**

0. Introduction

Virtualization techniques are essential for Cloud computing and have evolved significantly to address the challenges posed by complex architectures such as x86. We have studies/are studying para-virtualization, full virtualization and hardware-assisted virtualization techniques as part of this course.

KVM or Kernel-based Virtual Machine provides a hosted-hypervisor solution along with native Linux as the host operating system. KVM is implemented as a Linux kernel module and exposes a device (`/dev/kvm`) which are used by user-space programs (hypervisors) to perform operations through `ioctl()` calls – creation and managing of VMs.

The KVM API allows userspace hypervisors to perform the following operations:

- Creation of new virtual machines



cs695-assignment2

Updated automatically every 5 minutes

- ... and more.

References:

- KVM Paper: [\[link1\]](#)
- Linux KVM API: [\[link2\]](#) [\[link3\]](#)

While the userspace program of the hypervisor relies on KVM for the x86 architecture virtualization, it has to implement its own IO handling and device support — block IO, network IO, console drivers, USB controller, host file system, etc. In practice, QEMU is used as a userspace program alongside KVM (qemu-kvm), to setup and manage VMs.

KVM installation and usage – [link1](#)
[link2](#) [link3](#)

Note: Running KVM inside a VM requires nested virtualization. Support for non-x86 platforms has not been fully tested.

The assignment started code tarball – [link](#) (needs IITB LDAP)

Bonus: Use qemu-kvm to set up a Linux-based VM.

1a. DIY Hypervisor

In this assignment, we will build a simple hypervisor, set up simple VMs and control their execution, all with the help of KVM.

Starter code for a simple hypervisor is available as part of the assignment tarball at [assignment2/part1/](#)

The simple-kvm hypervisor is implemented in the source file `simple-kvm.c`. Since our hypervisor is simple, we cannot run a full-fledged OS. Therefore, the guest OS in `guest.c` is a single stream of instructions, with no separation between the guest user space and the guest OS.

cs695-assignment2

Updated automatically every 5 minutes

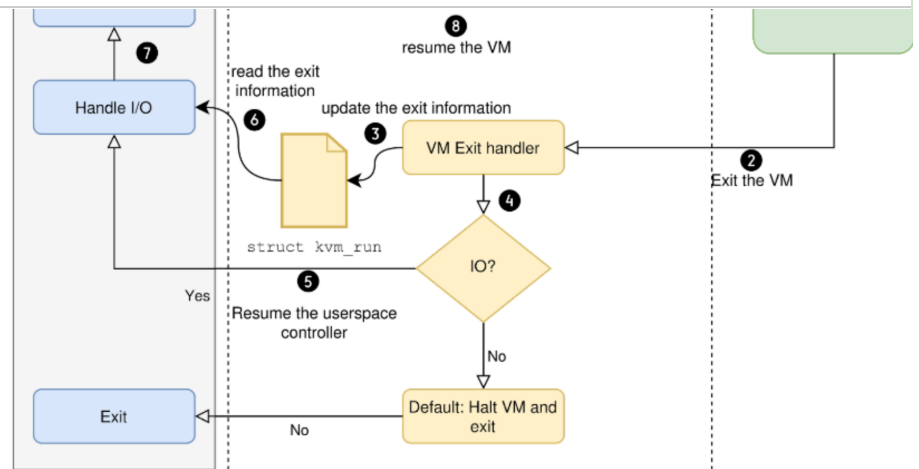


Figure 1: The basic VM run Loop of `simple-kvm.c`

The userspace hypervisor program **simple-kvm.c** uses the `/dev/kvm` file to create a virtual machine, allocate memory, and control CPU usage of the VM. The program then starts a KVM run loop that starts a vCPU using `KVM_RUN` and continuously handles the `VM_EXIT` routines when the vCPU returns from `KVM_RUN`, as shown in Figure 1. A simple print character operation by a guest translates to a write on port `0xE9` as a hypercall. The hypercall implementation for this setup uses the write IO operation to trigger a guest OS `VM_Exit`, and the exit cause is saved. The hypervisor checks that the VM has exited, trying to do an `IO_OUT` operation on port `0xE9`, and then the userspace handler of the hypervisor prints the value written to the port.

The x86 architecture has different operating modes for the CPU and the memory — real mode, protected mode, paged 32-bit, and long mode/paged 64-bit mode. Among other things, these modes refer to how the CPU will perform address translation. The **simple-kvm.c** program can create and operate VMs in any mode (mode selection is via a command line argument).

1. Go through the sample code and usage of KVM APIs in **simple-kvm.c** to understand working of the sample hypervisor. With the help of a well-drawn, labeled, and descriptive **flowchart**, explain the logical actions to setup and execute a VM in the **protected mode of operation**.

Note that this step needs a flowchart diagram; no other output will be considered valid.



cs695-assignment2

Updated automatically every 5 minutes

```
(*)
extern const unsigned char
guest64[], guest64_end[];
```

(b)

```
pml4[0] = PDE64_PRESENT | PDE64_RW
| PDE64_USER | pdpt_addr;
pdpt[0] = PDE64_PRESENT | PDE64_RW
| PDE64_USER | pd_addr;
pd[0] = PDE64_PRESENT | PDE64_RW |
PDE64_USER | PDE64_PS;
```

```
sregs->cr3 = pml4_addr;
sregs->cr4 = CR4_PAE;
sregs->cr0 = CR0_PE | CR0_MP |
CR0_ET | CR0_NE | CR0_WP | CR0_AM
| CR0_PG;
sregs->efer = EFER_LME | EFER_LMA;
```

(c)

```
vm->mem = mmap(NULL, mem_size,
PROT_READ | PROT_WRITE,
MAP_PRIVATE |
MAP_ANONYMOUS |
MAP_NORESERVE, -1, 0);
madvise(vm->mem, mem_size,
MADV_MERGEABLE);
```

(d)

```
case KVM_EXIT_IO:
if (vcpu->kvm_run->io.direction ==
KVM_EXIT_IO_OUT &&
vcpu->kvm_run->io.port == 0xE9)
{
char *p = (char *)vcpu-
>kvm_run;
fwrite(p + vcpu->kvm_run-
>io.data_offset,
vcpu->kvm_run-
>io.size, 1, stdout);
fflush(stdout);
continue;
}
```

(e)

```
memcpy(&memval, &vm-
>mem[0x400], sz);
```

Bonus: If you want to learn more about a full hypervisor implementation, you can refer to `kvmtool`, a lighter hypervisor than QEMU. <https://github.com/kvmtool/kvmtool>

Notes about the Makefile

- `simple-kvm` the hypervisor depends on two object files --- `simple-kvm.o` and `payload.o`
- `payload.o` is an object file that contains code from several binary files -



cs695-assignment2

Updated automatically every 5 minutes

labels that can be used as symbols to be referred to in the linked output binary. E.g., the labels `guest16`, `guest16_end`, `guest32`, `guest32_end`, `guest64`, `guest64_end` are used in `simple-kvm.c`

1b. Hypercalls: How can I help you?

Hardware-assisted hypercalls allow a Guest OS to communicate with the hypervisor. Since a Guest OS cannot directly perform any I/O operations, it exits to the hypervisor. The user space hypercall handler can then determine the exit reason and direction of the data and handle it correspondingly.

Note: The boilerplate code already implements an example hypercall `HC_print8bit()`; which writes a byte value to the port `0xE9` using assembly instruction `outb`.

Implement the following hypercalls to extend `simple-kvm.c` to be not-so-simple!

1. **`void HC_print32bit(uint32_t val)`**
This hypercall takes a 32-bit value as an argument and passes it to the hypervisor. The hypervisor's hypercall handler displays the 32-bit value on the terminal. Make sure to use a newline character after printing the value inside the hypervisor.
2. **`uint32_t HC_numExits()`**
The hypercall returns the number of times the guest VM has exited to the hypervisor from the beginning of its execution. The hypervisor should maintain the count and return it to the guest. Since the guest cannot print directly on the terminal, use `HC_print32bit()`, implemented in the previous step, to print the count.
3. **`void HC_printStr(char *str)`**
The hypervisor should handle this hypercall by printing the string pointed by `str`. Unlike the example code given in `guest.c`, which passes bytes of a string to `HC_print8bit`, the `HC_printStr` hypercall should make only one exit to the hypervisor. To achieve this, we must pass the address of the string to the hypervisor, which will fetch the



cs695-assignment2

Updated automatically every 5 minutes

You should not add any extra new lines. If the string passed by the guest has a new line, it should be printed.

The output of simple-kvm should show the difference of two exits (one for **HC_numExits** and one for **HC_printStr**)

4. **char* HC_numExitsByType()**

The hypervisor should handle the hypercall by returning an address to a string of the following format:

IO in: x

IO out: y

The values **x** and **y** should be the actual values of VM exits due to IO in and IO out. The **char*** value should then be passed to **HC_printStr** for printing via hypercall.

5. **uint32_t HC_gvaToHva(uint32_t gva)**

The hypercall should return the Host Virtual Address (HVA) corresponding to a given Guest Virtual Address (GVA). If the guest asks to translate an invalid GVA for which no HVA exists, the hypervisor should print "Invalid GVA" on the terminal and return 0. Demonstrate at least one valid and one invalid GVA to HVA conversion. Use **HC_print32bit** to print the HVA value.

Note: Actual HVA is more than 32 bits, but the guest has only 2MB of memory mapped. So the 32 bit LSB of HVA will only vary across mapped addresses. So you can assume loss of information and work with the truncated 32 bits address.

Hint: Lookup usage of the **KVM_TRANSLATE** request flag and **ioctl** with the **vmfd**. - **kvm_run** structure is heavily used in hypercalls.

outl and **in** assembly instructions should be used for writing and reading 32-bit values from a serial port.

Note: To implement multiple hypercalls, you may use a different serial port for each hypercall or a struct with a field with a hypercall number and all arguments to the hypercall. In the second method, the struct will be known to the guest, and the hypervisor and the guest will pass the address to the struct for every hypercall with the correct hypercall number and the arguments set. The hypervisor can read the hypercall number and the argument from the guest's memory. The implementation choice is left to you.



cs695-assignment2

Updated automatically every 5 minutes

2.0 Warmup

The starter code has `emu.c`, a custom KVM-based hypervisor, which sets up and runs two programs— `guesta.s` and `guestb.s`, inside their respective VMs. Each VM is set up with one vCPU each, and the startup code can configure them to run concurrently (via the Linux scheduler, scheduling each of the vCPU (pthread) threads independently).

Note that `emu.c` is the control program and cannot proactively pre-empt or de-schedule the VMs. It has to wait for VM_EXITs for control to reach back to it. The guest VMs have the following functionalities —

- **guesta.s**
Issues a hypercall (using port 0x10) with the value in the `ax` register, and then on return from hypercall increments the value stored in the `ax` register and reissues the hypercall, and does this in an infinite loop.
- **guestb.s**
Issues a hypercall (using port 0x11) with the value in the `ax` register, and then on return from hypercall decrements the value stored in the `ax` register and reissues the hypercall,, and does this in an infinite loop.

The size of the `guesta.s` and `guestb.s` is 7 bytes, which will be printed when the guest program is loaded in the user memory. Note that both VMs execute in **real** mode.

Note: The userspace program uses **pthread**s to handle VM exits in separate user threads. These threads are not to be confused with per vCPU threads that Linux uses for scheduling the VMs.

```
$ ./emu
VMFD: 4, Loaded
Program with size: 7
VMFD: 5, Loaded
Program with size: 7
VMFD: 4 started running
VMFD: 5 started running
VMFD: 4 stopped
running - exit reason: 2
VMFD: 4
KVM_EXIT_IO
VMFD: 4 out port: 16,
data: 0
```

The output shows the output of two VMs running concurrently.

The output is slightly controlled with a sleep statement in the user program. Otherwise, the output would have been even



cs695-assignment2

Updated automatically every 5 minutes

VMFD: 5 started running VMFD: 4 stopped running - exit reason: 2 VMFD: 4 KVM_EXIT_IO VMFD: 5 stopped running - exit reason: 2 VMFD: 5 KVM_EXIT_IO VMFD: 5 out port: 17, data: 65535 VMFD: 4 out port: 16, data: 1 VMFD: 5 started running VMFD: 4 started running VMFD: 5 stopped running - exit reason: 2 VMFD: 5 KVM_EXIT_IO VMFD: 5 out port: 17, data: 65534 VMFD: 4 stopped running - exit reason: 2 VMFD: 4 KVM_EXIT_IO VMFD: 4 out port: 16, data: 2	these x,y will be the same throughout a single run. But will stay consistent in runtime; this is applicable in all examples below]
--	--

2.1 Sharing a CPU

For our assignment, **emu** has to use only one physical CPU for its VMs and hence the vCPU threads have to share and execute on the same core (one at a time). To achieve this, **emu** has to support the following —

Instead of creating two pthreads for **vCPUs** (and one for the main thread, totaling three. You can observe the same using **htop**), modify the **kvm_run_vm** function to not make any pthread calls, but instead use the main thread to alternate between the two VMs on a **KVM_EXIT_IO**. This ensures that you can run two VMs on one physical CPU.

Test the implementation to check if your output is similar to that on 2.0.

Next, instead of just alternating between the two VMs, they must communicate with each other in a producer-consumer fashion. VM1 produces a number which has to be consumed by VM2. You have been given two guest programs with the following functionalities:

- **guest1a.s**
Issues a hypercall (using port 0x10) with the value in the **ax** register, and then on return from hypercall



cs695-assignment2

Updated automatically every 5 minutes

hypercall is stored in the **ax** register. This is followed by another hypercall to write the value from the **ax** register on port 0x12, and these steps are repeated in an infinite loop.

- **emu1.c** Complete this hypervisor implementation to the guests to communicate with each other as shown in the output below.

```
$ ./emu1
VMFD: 4 Loaded
Program with size: 7
VMFD: 5 Loaded
Program with size: 8
VMFD: 4
KVM_EXIT_IO
VMFD: 4 Produced
value: 0
VMFD: 5
KVM_EXIT_IO
VMFD: 5 Consuming
the number
VMFD: 5
KVM_EXIT_IO
VMFD: 5 Consumed
value: 0
VMFD: 4
KVM_EXIT_IO
VMFD: 4 Produced
value: 1
VMFD: 5
KVM_EXIT_IO
VMFD: 5 Consuming
the number
VMFD: 5
KVM_EXIT_IO
VMFD: 5 Consumed
value: 1
VMFD: 4
KVM_EXIT_IO
VMFD: 4 Produced
value: 2
VMFD: 5
KVM_EXIT_IO
VMFD: 5 Consuming
the number
VMFD: 5
KVM_EXIT_IO
VMFD: 5 Consumed
value: 2
VMFD: 4
KVM_EXIT_IO
VMFD: 4 Produced
value: 3
VMFD: 5
KVM_EXIT_IO
```

The single-threaded execution model ensures that the VMs run one after another.

Note that, in this example, each time VM1 is scheduled to produce a value, VM2 has to be scheduled twice to consume and verify the consumption.

All print statements are by the hypervisor.



cs695-assignment2

Updated automatically every 5 minutes

2.2 Bursty scheduling

For this part, we will use the guest programs **guest2a.s** and **guest2b.s**. Further, instead of sharing the CPU one after the other, the VMs are to be scheduled to consume the CPU in a bursty fashion. Specifically, VM1, the producer VM produces three numbers before the second VM gets a chance to consume them, the VM1 produces 3 items again and the cycle repeats. Note that VM1 produces 1 item per hypercall.

Modify the file **emu2.c** to enable this functionality.

A sample output is as follows.

```
$ ./emu2
VMFD: 4 Loaded
Program with size: 7
VMFD: 5 Loaded
Program with size: 8
VMFD: 4
KVM_EXIT_IO
VMFD: 4 Produced
value: 0
VMFD: 4
KVM_EXIT_IO
VMFD: 4 Produced
value: 1
VMFD: 4
KVM_EXIT_IO
VMFD: 4 Produced
value: 2
VMFD: 5
KVM_EXIT_IO
VMFD: 5
KVM_EXIT_IO
VMFD: 5 Consumed
value: 0
VMFD: 5
KVM_EXIT_IO
VMFD: 5
KVM_EXIT_IO
VMFD: 5 Consumed
value: 1
VMFD: 5
KVM_EXIT_IO
VMFD: 5
KVM_EXIT_IO
VMFD: 5 Consumed
value: 2
VMFD: 4
KVM_EXIT_IO
```

Note that, in this example, VM1 is being scheduled 3 times to produce 3 values, VM2 has to be scheduled accordingly to consume 3 values and verify the consumption.

All print statements are by the hypervisor.



cs695-assignment2

Updated automatically every 5 minutes

```

KVM_EXIT_IO
VMFD: 4 Produced
value: 5
VMFD: 5
KVM_EXIT_IO
VMFD: 5
KVM_EXIT_IO
VMFD: 5 Consumed
value: 3
VMFD: 5
KVM_EXIT_IO
VMFD: 5
KVM_EXIT_IO
VMFD: 5 Consumed
value: 4
VMFD: 5
KVM_EXIT_IO
VMFD: 5
KVM_EXIT_IO
VMFD: 5 Consumed
value: 5
....

```

2.3 Mass production

Now we want to do mass production. So instead of producing and consuming a single number in each hypercall, we **produce and consume five numbers at a go with each hypercall**. For the same purpose, two **incomplete** guest programs: **guest3a.c** and **guest3b.c** have been provided. You are free to reuse the hypercalls implemented in **part 1** or create new hypercalls for these guest programs as per your convenience.

UPDATED INSTRUCTIONS: The producer VM produces 5 items in an array without trapping to the hypervisor. After production, the producer VM traps and passes the base address of the array to the hypervisor. The hypervisor copies buffer content to the ConsumerVM's array and prints the produced values. Finally, the hypervisor schedules the Consumer VM, which after consumption traps to the hypervisor with the number of elements consumed and the elements themselves. The hypervisor uses this information to print the consumed values and the process repeats.

Note that both the VMs now execute in **protected mode**. Modify the **run_vm** function in **emu3.c**, along with necessary changes in the guest programs to obtain functionality similar to as shown in the output below:

```

$ ./emu3
VMFD: 4 Loaded Program with size: 236

```



cs695-assignment2

Updated automatically every 5 minutes

```

VMFD: 4 Produced Values: 15 16 17 18 19
VMFD: 6 Consumed Values: 15 16 17 18 19
VMFD: 4 Produced Values: 20 21 22 23 24
...

```

2.4 The not-so-deterministic world

Let's get real, at least a bit — this world is non-deterministic. So is the case with our producer and consumer VMs, who wish to produce and consume a random number of elements on each hypercall instead of the fixed number as required in **part 2.3**.

Modify/complete the **run_vm** function in **emu4.c** and the guest programs **guest4a.c** and **guest4b.c**, so that the following properties can be achieved:

- The hypervisor coordinates between the VMs to provide a virtual shared buffer of a finite size (default is 20 elements). Each of the VMs maintains a local copy of the buffer to either produce or consume elements. Once the VM completes its action, it conveys the state of the buffer (producer and consumer pointers) to the hypervisor, which uses this information to copy the buffer to the other VM's buffer (if required) and update its own state of the buffer. The state of the buffer (producer and consumer pointers) are read by the VMs via a hypercall and then a corresponding action executed (production/consumption).
- **prod_p** points to the location where the last produced element was stored.
- **cons_p** points to the location of the next item to be consumed.
- The producer and consumer VMs communicate the values of **prod_p** and **cons_p** pointers between each other (via the hypervisor) to know the boundaries of the buffer within which they have to produce and consume respectively.
- The producer does not cause overflow and the consumer does not cause underflow in the buffer.
- The number of items produced or consumed in one go are in the range **[0, 10]**. Each VM picks a random number in this range for action during each scheduling iteration.

one iteration of produce or consume.

- Each VM when "scheduled" should read producer-consumer buffer state, act on it and convey the new state back to the hypervisor.

Note:

To generate random numbers in guest VMs the [rdtsc](#) assembly instruction can be used. Check how to use it via inline assembly in C.
The state of the buffer has to be conveyed to the VMs explicitly so that they do not overflow or underflow the buffer. Similarly, the state of the buffer after production or consumption has to be conveyed from the respective VMs to the hypervisor.

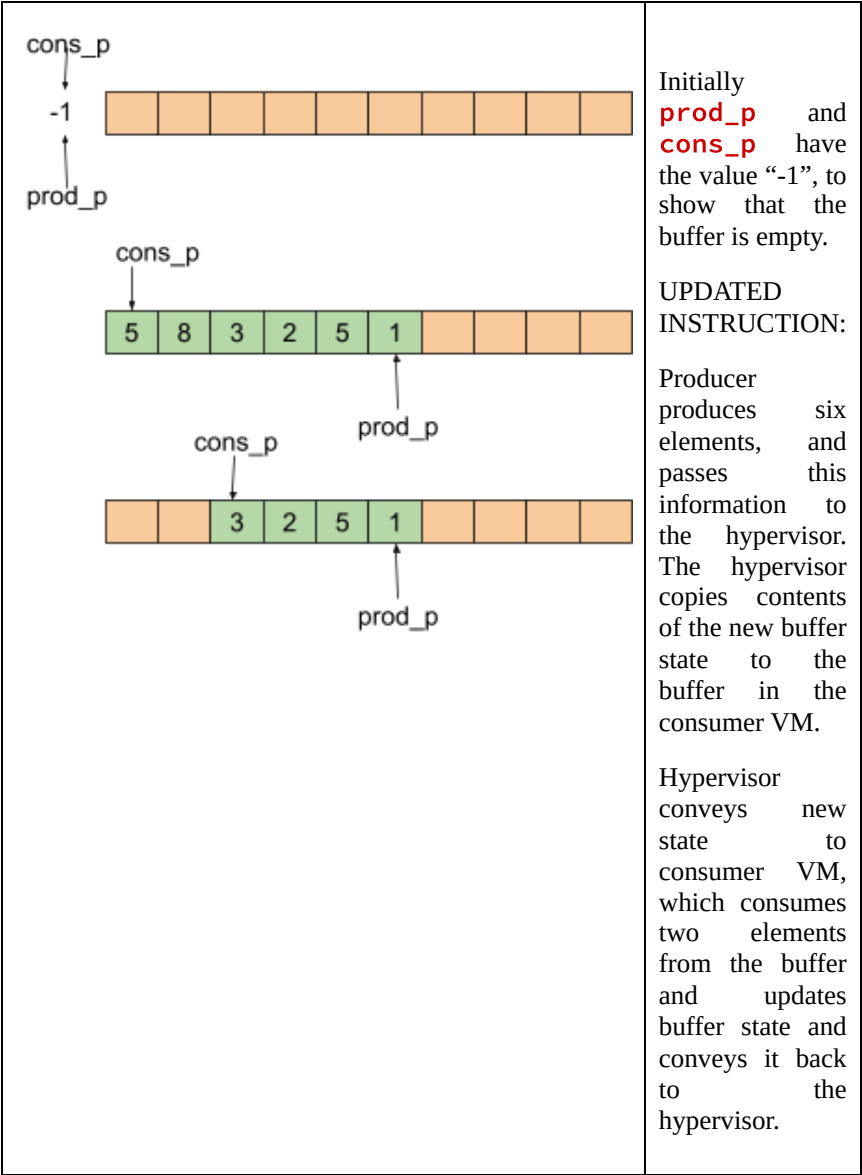
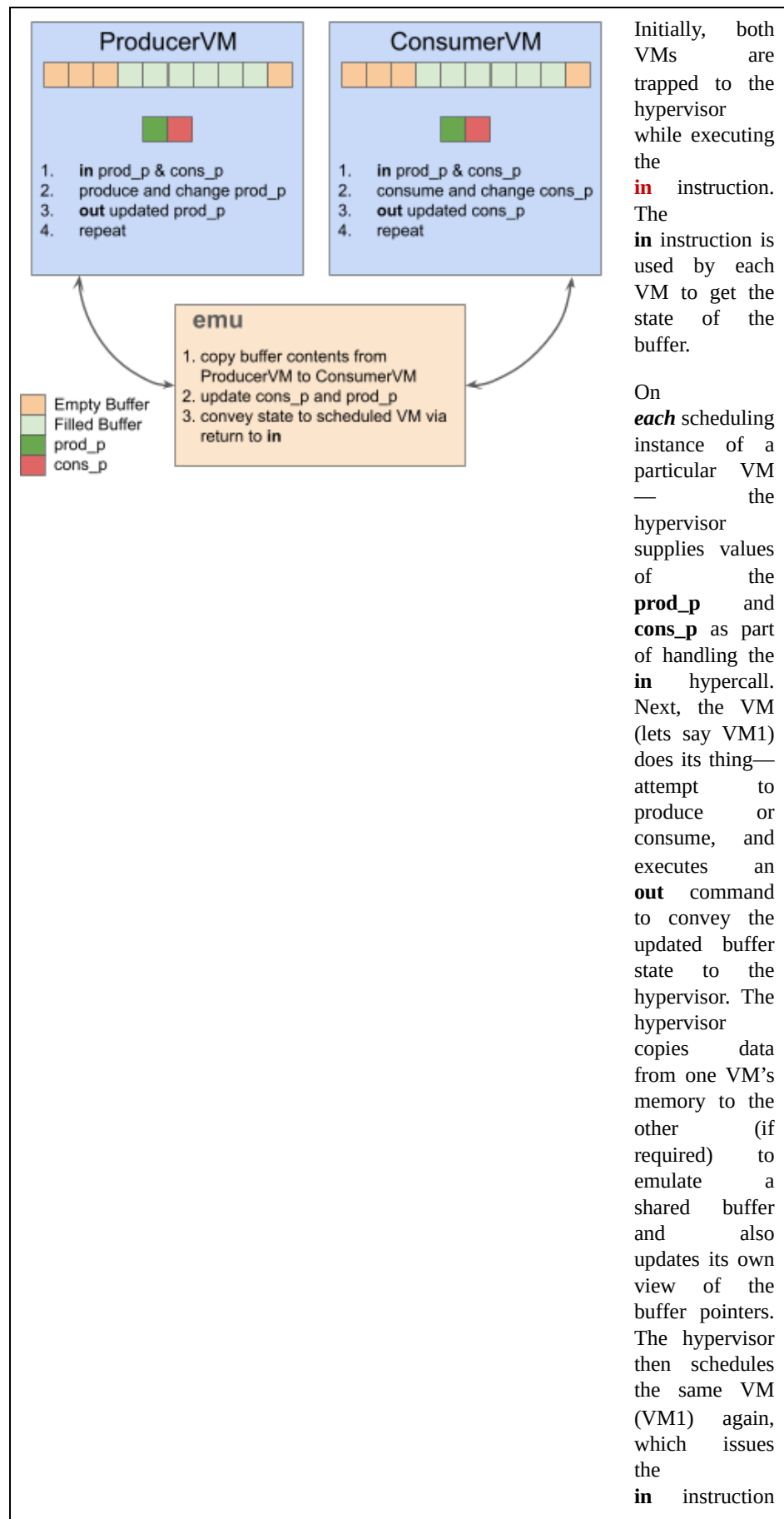


Figure 2: Example working of non-deterministic producer-consumer sequence,

cs695-assignment2

Updated automatically every 5 minutes

NUMBER OF ELEMENTS FROM THE BUFFER.





cs695-assignment2

Updated automatically every 5 minutes

A sample output is as shown below:
UPDATED OUTPUT

```
$ ./emu4 sched1.txt
VMFD: 4 Loaded Program with size: 236
VMFD: 6 Loaded Program with size: 252
HYPVSR: []
VMFD: 4 Produced 5 Values: 8 6 4 2 3
HYPVSR: [8 6 4 2 3]
VMFD: 6 Consumed 3 Values: 8 6 4
HYPVSR: [2 3]
VMFD: 4 Produced 2 Values: 6 5
HYPVSR: [2 3 6 5]
VMFD: 6 Consumed 1 Values: 2
HYPVSR: [3 6 5]
VMFD: 4 Produced 4 Values: 5 3 1 9
HYPVSR: [3 6 5 5 3 1 9]
VMFD: 6 Consumed 3 Values: 3 6 5
HYPVSR: [5 3 1 9]
...
```

Submission instructions

- All submitted code should be well-commented and readable.
- Submit a **PDF** file for Part 1a [named **part1a.pdf**] and final code solutions for Parts 1b and 2 according to the directory structure. Make sure that you execute **make clean** before submitting solutions.
- Name your submission folder as `<rollnumber>_assignment2` and the tarball as `<rollnumber>_assignment2.tar.gz` (e.g. `22d0999_assignment2.tar.gz`) - all small letters

```
tar -czvf <rollnumber>_assignment2.tar.gz <rollnumber>_assignment2
```
- The tar should contain the following files in the following directory structure:

```
<rollnumber>_assignment2/
├── .git/
│   └── .../* all git-related
files */
├── part1a.pdf
├── part1/
│   ├── guest.c
│   ├── guest.ld
│   ├── guest16.s
│   ├── Makefile
│   ├── payload.ld
│   ├── README.md
│   └── simple-kvm.c
```



Published using Google Docs

[Report abuse](#)[Learn more](#)

cs695-assignment2

Updated automatically every 5 minutes

```
graph LR
    Root[ ] --- guest3b.c
    Root --- guest4a.c
    Root --- guest4b.c
    Root --- emu.c
    Root --- emu1.c
    Root --- emu2.c
    Root --- emu3.c
    Root --- emu4.c
    Root --- payload.ld
    Root --- guest.ld
    Root --- Makefile1[Makefile]
    Root --- README.md1[README.md]
    Root --- .gitignore
    Root --- Makefile2[Makefile]
    Root --- README.md2[README.md]
```

Please strictly adhere to this format; else, your submission will not count.

~~22d0999_assignment2.gz~~ ~~22d0999_assignment2.zip~~ ~~22d0999_assignment2.tar.xz~~

- Before submission, make sure the tarball is not corrupted by opening it. Please keep the .git folder in your submission that comes along with the starter code. Modify the README.md files to explain your submission.

**Deadline: 12 Feb 2025,
11:59 PM via Moodle**